2026-5-27 C++多态

- 1. 多态的概念
  - 1.1 编译时多态（静态多态）
  - 1.2 运行时多态（动态多态）
- 2. 多态的定义以及实现
  - 2.1 多态的构成条件
    - 2.1.1 实现多态的两个条件
    - 2.1.2 虚函数
    - 2.1.3 虚函数的重写/覆盖
    - 2.1.4 多态场景下的一个小测试
    - 2.1.5 虚函数重写的一些其他问题
    - 2.1.6 override和final
    - 2.1.7 重载/重写/隐藏的对比
- 3. 纯虚函数和抽象类
- 4. 多态原理
  - 4.1 虚函数表指针
  - 4.2 多态的原理
    - 4.2.1 多态是如何实现的
    - 4.2.2 动态绑定与静态绑定
    - 4.2.3 虚函数表

1. 多态的概念

多态实际上就是多种形态，多态分为两种：

1.1 编译时多态（静态多态）

编译时多态主要时多态主要就是我们之前提到的函数重载和函数模板，他们传不同类型的参数就可以调用不同的函数，通过参数不同达到多种形态，之所以叫做编译时多态是因为他们实参传递给形参的参数匹配是在编译时完成的，我们把编译时一般归为静态。

1.2 运行时多态（动态多态）

运行时多态实际上就是去完成某个行为（函数），可以传不同的对象就会完成不同的行为，就达到多种形态。比如说我们坐火车买票这个行为，普通人买票时是全价买票；学生买票时是半价买票;军人买票时优先买票。再比如说同样是动物的一个行为（函数），传对象猫过去就是

“喵喵”，传对象狗过去就是“汪汪”。
举一个简单的例子：

```
class Person {
public:
    virtual void BuyTicket() {
        cout << "买票全价" << endl;
    }
};
class Student :public Person {
    virtual void BuyTicket() {
        cout << "买票半价" << endl;
    }
};
void Func(Person& people) {
    people.BuyTicket();
}
int main() {
    Person ZhangSan;
    Func(ZhangSan);

    Student WangWu;
    Func(WangWu);

    return 0;
}
```

2. 多态的定义以及实现

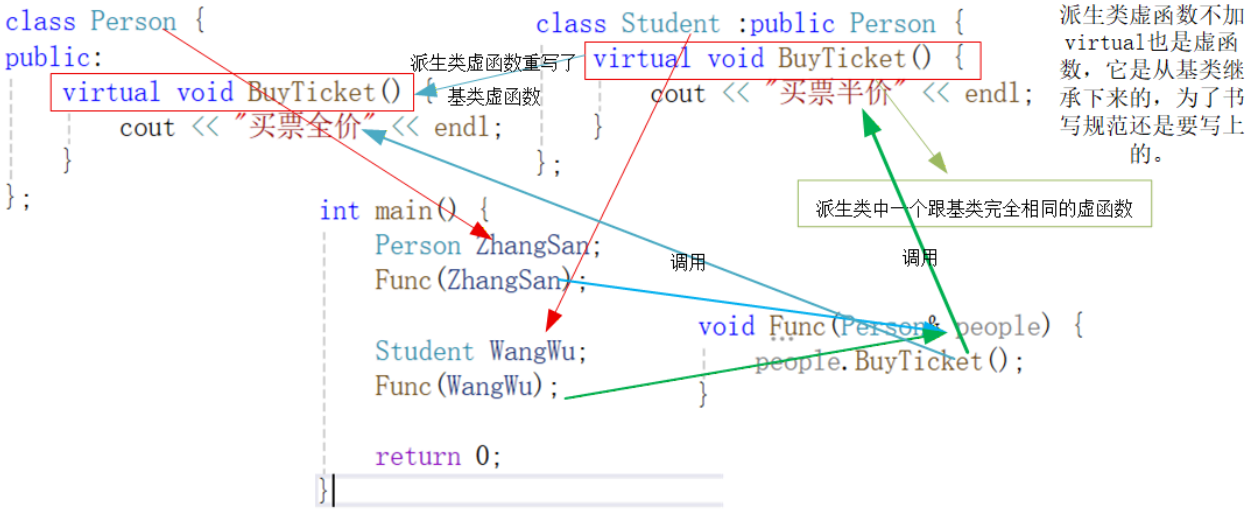
2.1 多态的构成条件

多态是一个继承关系下的类对象，去调用同一个函数，产生了不同行为。比如所Student继承了Person.Person对象买票全价格，Student对象优惠买票。

2.1.1 实现多态的两个条件

1. 必须是基类的指针或者引用调用虚函数

2. 被调用的函数必须是虚函数，并且完成了虚函数的重写/覆盖。



> 要实现多态的效果有两个重要的点：
>
> 1. 必须的是基类的指针或者引用做参数，因为只有基类的指针或者引用才能既指向基类对象又指向派生类对象。
> 2. 派生类对象必须对基类的虚函数完成重写/覆盖，重写或者覆盖了，基类和派生类之间才能有不同的函数，多态的效果才能达到。

2.1.2 虚函数

类成员前面加virtual修饰，那么这个成员函数被称为虚函数。注意非成员函数不能加virtual修饰。

```
class Animal {
public:
    virtual void speak() const
    {            // 正确：成员虚函数
        cout << "动物在叫:" << endl;
    }
};
class Dog : public Animal {//派生类
public:
    virtual void speak() const
    {           // 重写虚函数,加virtual修饰
        cout << "汪汪汪！" << endl;
    }
};
class Cat : public Animal {//派生类
public:
    virtual void speak() const
```

```
    {           // 重写虚函数
        cout << "喵喵喵！" << endl;
    }
};
//非成员函数不能加 virtual，会出现编译错误
//virtual void printHello() {
//    cout << "Hello" << endl;
//}

void letsHear(const Animal& animal) {
    animal.speak();
}

int main() {
    Cat cat;
    Dog dog;
    letsHear(cat);
    letsHear(dog);
    return 0;
}
```

2.1.3 虚函数的重写/覆盖

虚函数的重写/覆盖：派生类中有一个跟基类完全相同的虚函数（即派生类虚函数与基类虚函数的返回值类型、函数名字、参数列表完全相同），称派生类的虚函数重写了基类的虚函数。

```
class Person {
public:
    virtual void BuyTicket() {
        cout << "买票全价" << endl;
    }
};
class Student :public Person {
    virtual void BuyTicket() {
        cout << "买票半价" << endl;
    }
};
void Func(Person* ptr) {
    //这里可以看到虽然都是Person指针在调用BuyTicket
    //这个东西本质上是与ptr指向的对象有关
    ptr->BuyTicket();
}
int main() {
    Person ZhangSan;
```

```
    Func(&ZhangSan);

    Student WangWu;
    Func(&WangWu);

    return 0;
}
```

> 需要注意的是：在重写基类虚函数时，派生类的虚函数在不加virtual关键字时，虽然也可以构成重写（因为继承后基类的虚函数被继承下来了在派生类中依旧保持虚函数属性）这样的写法是不规范的，不建议这样使用。但是在考题中，经常会故意埋个坑，让你判断是否构成多态。

2.1.4 多态场景下的一个小测试

下列程序的输出结果是什么？( )
A.A->0 B.B->0 C:A->1 D:B->0 E:编译出错 F：以上都不正确

```
class A
{
public:
    virtual void func(int val = 1) { cout << "A->" << val << endl; }
    virtual void text( ) { func(); }
};
class B :public A
{
public:
    void func(int val = 0){ cout << "B->" << val << endl; }//继承虚函数构成多
态
};
int main(int argc,char* argv[]) {
    B* p = new B;
    p->text();
    return 0;
}
```

派生类类的指针调用text,将`B*`的this指针传递给text,基类和派生类中的两个func()函数构成多态，所以这里的func()调用的是B类中的func。最最关键的在于，构成重写的两个虚函数，重写的是虚函数的实现，所以实际上val的值为1，用的B中func.
继承并不是将基类的成员拷贝一份到派生类中，它是分两部分检索的，要不然前面的隐藏就没有意义了，会直接构成函数重载。

2.1.5 虚函数重写的一些其他问题

1. 协变
   派生类重写基类虚函数时，与基类虚函数返回值类型不同。即基类虚函数返回基类对象的指针或者引用时称为协变。（意义不大）
   协变的用途：

用途	说明
工厂方法模式	基类返回 `Product*`，派生类返回 `ConcreteProduct*`，避免外部向下转型
克隆（Clone）模式	`virtual Base* clone() = 0;`，派生类返回自己的类型，保持类型安全
链式调用	派生类成员函数返回自身引用或指针，支持派生类特有的链式操作
避免显式向下转型	基类接口返回基类指针，派生类实现返回派生类指针，调用方可直接使用派生类接口

```
class A{};
class B:public A{};
class Person {
public:
    //基类虚函数返回基类的指针或引用
    virtual A* BuyTicket() {
        cout << "买票全价" << endl;
    }
};
class Student :public Person {
    //派生类对象返回派生类的指针或引用
    virtual B* BuyTicket() {
        cout << "买票半价" << endl;
    }
};
void Func(Person* ptr) {
    ptr->BuyTicket();
}
int main() {
    Person ps;
    Student st;

    Func(&ps);
    Func(&st);
```

```
    return 0;
}
```

1. 析构函数的重写
   基类的析构函数称为虚函数，此时派生类析构函数只要定义，无论是否加virtual关键字，都与基类的析构函数构成重写，虽然基类与派生类析构函数名不同看起来不符合重写规则，实际上编译器对析构函数做了特殊处理，编译后析构函数的名称统一处理成destructor，所以基类的析构函数加了viatual修饰，派生类的析构函数就构成重写。

```
class A {
public:
    virtual ~A() {
        cout << "~A()" << endl;
    }
};
class B :public A {
public:
    ~B() {
        cout << "~B->delete:" << _p << endl;
        delete[] _p;
    }
protected:
    int* _p = new int[10];
};
//只有派生类Studentde的析构函数重写了Person的析构函数，下面的delete对象调用析构函数才能构成多态
//才能保证p1和p2指向的对象正确的调用析构函数
int main() {
    A* p1 = new A;
    A* p2 = new B;
    delete p1;
    delete p2;
    return 0;
}
```

通过将基类的析构函数声明为 `virtual`，可以确保在使用基类指针删除派生类对象时，正确调用派生类（以及基类）的析构函数，从而避免内存泄漏。
如果~A()不加virtual，那么delete p2只调用A的析构函数，没有调用B的析构函数就会导致内存泄漏，因为~B()中在释放资源。

2.1.6 override和final

我们可以得出，C++对虚函数重写的要求比较严格，但是有些情况下由于疏忽，比如函数名写错参数，写错无法构成重写，而这种错误在编译期间是不会爆出的。如果登到程序运行时再来debug这样的效率就会极低，因此C++11提供了override,可以帮助用户是否重写。如果我们不想让派生类重写这个虚函数，可以用final去修饰。

```
class Car {
public:
    virtual void Dirve() {

    }
};
class Benz :public Car {
public:
    virtual void Dirve() override { cout << "Benz" << endl; }
};
int main() {
    return 0;
}
```

如果不想被重写就加上final

```
class Car {
public:
    virtual void Dirve() final{

    }
};
class Benz :public Car {
public:
    virtual void Dirve() { cout << "Benz" << endl; }
};
int main() {
    return 0;
}
```

2.1.7 重载/重写/隐藏的对比

（常考）



3. 纯虚函数和抽象类

在虚函数的后面写上=0，则这个函数为虚函数，纯虚函数不需要定义实现（实现没啥意义因为要被派生类重写，但是语法上可以实现），只要声明即可。包含虚函数的类叫做抽象类，抽象类不能实例化出对象，如果派生类继承后不重写纯虚函数，那么派生类也是也是抽象类。纯虚函数某种程度上强制了派生类重写虚函数，因为不重写你就实例化不出对象。

```
class Car {
    public:
        virtual void Dirve() = 0;//纯虚函数，存在的是因为需要被重写
    };
    class Benz :public Car {
    public:
        virtual void Dirve() override { cout << "Benz" << endl; }
    };
    class BMW :public Car {
    public:
```

```
        virtual void Dirve() override { cout << "BMW" << endl; }
    };
    int main() {
        //Car car;无法实例化出对象
        Car* pBenz = new Benz;
        pBenz->Dirve();
        Car* pBMW = new BMW;
        pBMW->Dirve();
        return 0;
    }
```

4. 多态原理

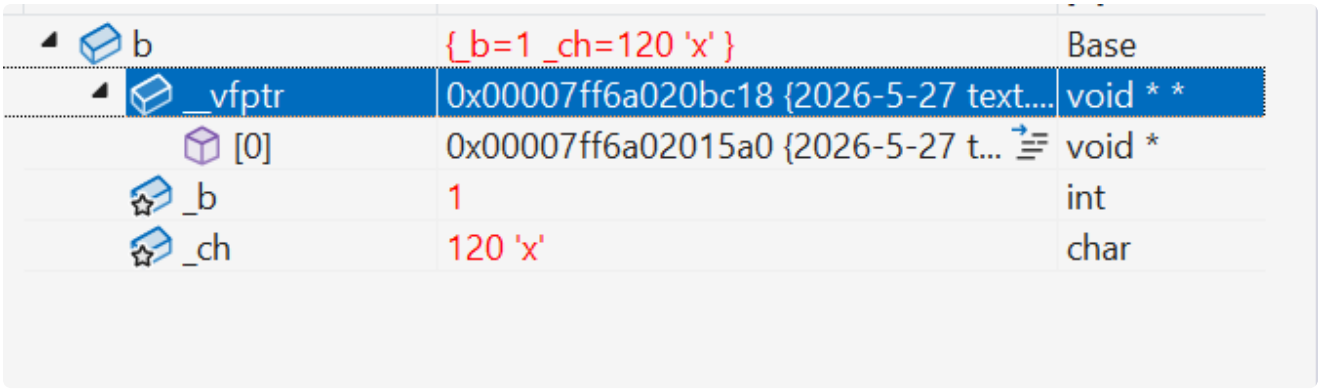
4.1 虚函数表指针

下面编译为32位程序的运行结果是()
A.编译报错 B.运行报错 C.8 D.12

```
class Base {
public:
    virtual void Func1() {
        cout << "Func1()" << endl;
    }
protected:
    int _b = 1;
    char _ch = 'x';
};
int main() {
    Base b;
    cout << sizeof(b) << endl;
    return 0;
}
```

运行结果是12，处理成员_b和_ch对象还多了一个_vfptr放在对象的前面（有的平台可能放在后面，这个和平台有关），对象中的这个指针我们叫虚函数表指针（v:virtual,f:function)。一个含有虚函数的类都至少有一个虚函数表指针，因为一个类所有虚函数的地址都要被放到这个类

对象的虚函数表中，虚函数表简称虚表。



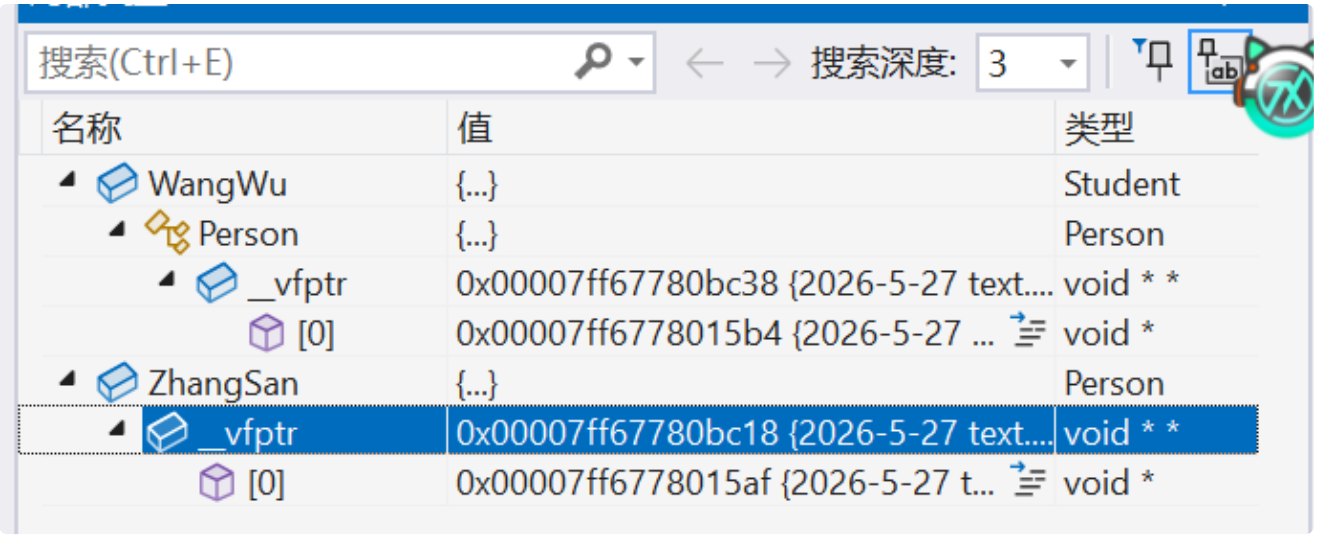
4.2 多态的原理

4.2.1 多态是如何实现的

从底层的角度Func函数中ptr->BuyTicket()，是如何作为ptr指向Person对象调用Person::BuyTicket,ptr指向Student对象调用Student::BuyTicket的呢？满足多态条件后，底层不再是编译时通过调用对象函数确定的地址，而是运行时到指向对象的虚表中确定对应的虚函数地址，这样就实现了指针或引用指向基类就调用基类的虚函数，指向派生类就调用派生类对应的虚函数。

```
class Person {
public:
    virtual void BuyTicket() {
        cout << "买票全价" << endl;
    }
};
class Student :public Person {
    virtual void BuyTicket() {
        cout << "买票半价" << endl;
    }
};
void Func(Person& people) {
    people.BuyTicket();
}
int main() {
    Person ZhangSan;
    Func(ZhangSan);
    Student WangWu;
    Func(WangWu);

    return 0;
}
```



4.2.2 动态绑定与静态绑定

对不满足多态条件（指针或者引用+调用虚函数）的函数调用是在编译时绑定，也就是编译时确定调用函数的地址，叫做静态绑定。
不满足多态条件的情况：不是通过指针/引用调用虚函数，或者调用的不是虚函数。

```
#include <iostream>
using namespace std;

class Animal {
public:
    void speak() {    // 非虚函数
        cout << "动物叫" << endl;
    }
    virtual void eat() {
        cout << "动物吃" << endl;
    }
};

class Dog : public Animal {
public:
    void speak() {
        cout << "汪汪汪" << endl;
    }
    virtual void eat() override {
        cout << "狗吃骨头" << endl;
    }
};

int main() {
    Dog d;
```

```
    Animal* p = &d;

    // 1. 通过对象调用（不是指针/引用）→ 静态绑定
    d.speak();    // 输出：汪汪汪（编译时已确定是 Dog::speak）
    d.eat();      // 输出：狗吃骨头（对象调用，编译时确定）

    // 2. 调用非虚函数 → 静态绑定
    p->speak();   // 输出：动物叫（非虚函数，编译时按指针类型 Animal* 绑定）

    return 0;
}
```

满足多态条件的函数调用是在运行时绑定，也就是在运行时到指向对象的虚函数表中找到调用函数的地址，叫动态绑定。

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void speak() {   // 虚函数
        cout << "动物叫" << endl;
    }
};

class Dog : public Animal {
public:
    virtual void speak() override {
        cout << "汪汪汪" << endl;
    }
};

class Cat : public Animal {
public:
    virtual void speak() override {
        cout << "喵喵喵" << endl;
    }
};

void makeSound(Animal* p) {   // 通过指针调用虚函数
    p->speak();                // 运行时确定调用哪个版本
}

int main() {
    Dog d;
```

```
    Cat c;

    makeSound(&d);   // 输出：汪汪汪（运行时通过 Dog 的虚表找到 Dog::speak）
    makeSound(&c);   // 输出：喵喵喵（运行时通过 Cat 的虚表找到 Cat::speak）

    return 0;
}
```

4.2.3 虚函数表

1. 基类对象的虚函数表中存放基类所有虚函数的地址。同类型的对象共用一张虚表，不同类型的对象各自有独立的虚表，所以基类和派生类有各自独立的虚表。
2. 派生类由两部分构成，继承下来的基类和自己的成员，一般情况下，继承下来的基类中有虚函数表指针，自己就不会再生成虚函数表指针。但是要注意的是，这里继承下来的基类部分虚函数表指针和基类对象的虚函数表指针不是同一个，就像基类对象的成员和派生类对象中的成员也独立的。
3. 派生类中重写的基类的虚函数，派生类的虚函数表中对应的虚函数就会被覆盖派生类重写的虚函数地址。

```
class Base
    {
    public:
        virtual void Func1()
        {
            cout << "Func1()" << endl;
        }

        virtual void Func2()
        {
            cout << "Func1()" << endl;
        }

        void Func3()
        {
            cout << "Func1()" << endl;
        }
    protected:
        int _b = 1;
        char _ch = 'x';
    };

    int main()
    {
```

```
        Base b;
        cout << sizeof(b) << endl;

        return 0;
    }
```

1. 派生类的虚函数表中包含：

- 基类的虚函数地址
- 派生类重写的虚函数地址完成覆盖，派生类自己的虚函数地址分为三个部分

1. 虚函数表本质上是一个存虚函 数指针的指针数组，一般情况这个数组最后面放一个0x00000000标记。（这个C++并没有做明确的规定，是各个编译器自定义的，vs系列编译器会在后面放个0x00000000标记，g++系列编译器不会放）
2. 虚函数存在哪里？虚函数和普通函数一样，编译好后是一段指令，都是存放在代码段的，只是虚函数的地址又存到了虚表中。
3. 虚函数表存到哪里？这个问题严格C++并没有严格规定。

```
class Person {
public:
    virtual void BuyTicket() {   // 虚函数
        cout << "Person: 买票全价" << endl;
    }
    virtual void Func1() {};
    void Func2() {};
private:
    string _name;
};

class Student : public Person {
public:
    virtual void BuyTicket() override {   // 重写虚函数
        cout << "Student: 买票半价" << endl;
    }
    virtual void Func1() {};
    void Func4() {};
private:
    string _id;
};

void Func(Person* ptr) {
    ptr->BuyTicket();   // 这里在运行时决定调用哪个版本
```

```
}


int main()
{
    int i = 0;
    static int j = 1;
    int* p1 = new int;
    const char* p2 = "xxxxxxxx";
    printf("栈:%p\n", &i);
    printf("静态区:%p\n", &j);
    printf("堆:%p\n", p1);
    printf("常量区:%p\n", p2);

    Person p;
    Person* ptr = &p;
    Student s;
    printf("虚表:%p\n", *((int*)ptr));
    printf("虚表:%p\n", *((int*)&s));

    printf("虚函数地址:%p\n", &Person::Func1);
    printf("普通函数地址:%p\n", &Person::Func2);

    return 0;
}
```